**Problem Statement**

Wine Type Classification Using Random Forest

Classify wines into one of three types using Random Forest, an ensemble of decision trees, to improve prediction accuracy and robustness. Random Forest reduces overfitting and improves generalization compared to a single Decision Tree.

**Step 0: Import Libraries**

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
```

**Step 1: Load Dataset**

```
wine = load_wine()
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = pd.Series(wine.target)

print(X.head())
print(y.value_counts())
```

```
   alcohol  malic_acid   ash  alcalinity_of_ash  magnesium  total_phenols  \
0    14.23        1.71  2.43               15.6      127.0           2.80
1    13.20        1.78  2.14               11.2      100.0           2.65
2    13.16        2.36  2.67               18.6      101.0           2.80
3    14.37        1.95  2.50               16.8      113.0           3.85
4    13.24        2.59  2.87               21.0      118.0           2.80

   flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity   hue  \
0        3.06                  0.28             2.29             5.64  1.04
1        2.76                  0.26             1.28             4.38  1.05
2        3.24                  0.30             2.81             5.68  1.03
3        3.49                  0.24             2.18             7.80  0.86
4        2.69                  0.39             1.82             4.32  1.04

   od280/od315_of_diluted_wines  proline
0                          3.92   1065.0
1                          3.40   1050.0
2                          3.17   1185.0
3                          3.45   1480.0
4                          2.93    735.0
1    71
0    59
2    48
Name: count, dtype: int64
```

**Step 2: Train-Test Split**

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

**Step 3: Feature Scaling (Optional for RF)**

Random Forest does not require feature scaling, but you can scale if you want to compare with other models.

**Step 4: Train Random Forest Classifier**

```
rf_model = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
rf_model.fit(X_train, y_train)
```

```
RandomForestClassifier(max_depth=5, random_state=42)
```

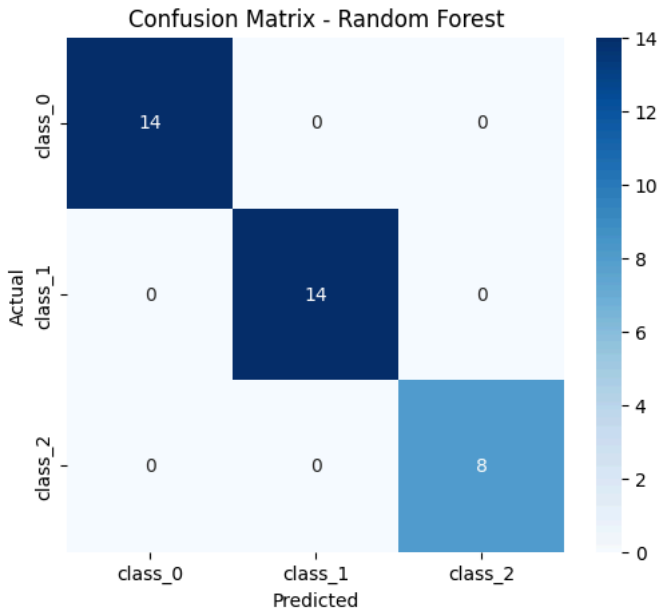
### Step 5: Make Predictions

```
y_pred = rf_model.predict(X_test)
```

### Step 6: Evaluate Model

```
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("Accuracy:", accuracy)
print(classification_report(y_test, y_pred))

# Confusion Matrix Visualization
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=wine.target_names, yticklabels=wine.target_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Random Forest')
plt.show()
```

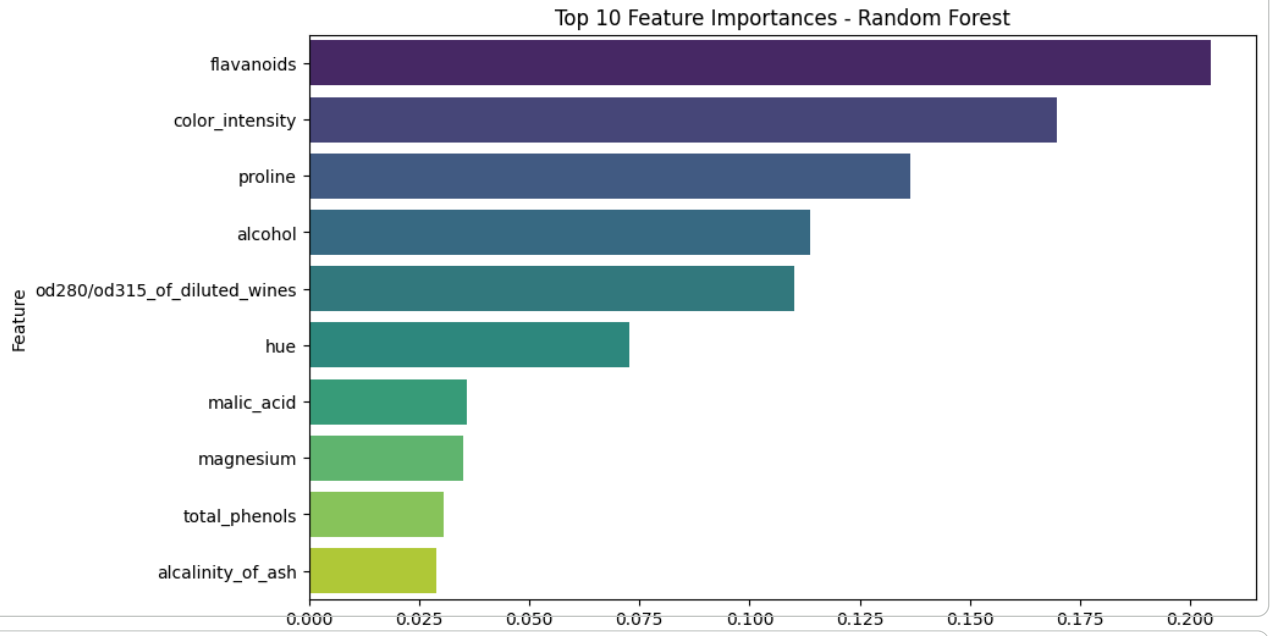
```
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        14
           1       1.00      1.00      1.00        14
           2       1.00      1.00      1.00         8

    accuracy                           1.00        36
   macro avg       1.00      1.00      1.00        36
weighted avg       1.00      1.00      1.00        36
```




### Step 7: Feature Importance Visualization

```
# Get feature importance
feat_imp = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
}).sort_values(by='Importance', ascending=False)

# Plot top 10 important features
plt.figure(figsize=(10,6))
sns.barplot(x='Importance', y='Feature', data=feat_imp.head(10), palette='viridis')
plt.title('Top 10 Feature Importances - Random Forest')
plt.show()
```

```
/tmp/ipython-input-2842067468.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and

  sns.barplot(x='Importance', y='Feature', data=feat_imp.head(10), palette='viridis')
```




```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(rf_model, X, y, cv=5)
print(scores.mean())
```

```
0.9720634920634922
```

**Classification Exercises Summary**

Dataset: sklearn.datasets.load_wine() Models Used:

Decision Tree (DT)

Random Forest (RF)

Cross Validation for generalization check

**Decision Tree Classifier (DT)**

DT learns decision rules by splitting features

Can easily overfit if tree grows deep

Results: Test Accuracy: 94%

**Interpretation:**

DT captures patterns well but:

Learns specific splits from training data

Sensitive to small data variations

A single tree = high variance model

Conclusion: DT performs well but may not generalize perfectly.

**Random Forest Classifier (RF): **

Ensemble of multiple Decision Trees

Results: Test Accuracy: 100%

Cross-Validation Accuracy: 97%

Interpretation

RF benefits from:

Averaging many trees

Important: 100% accuracy does NOT automatically mean overfitting so verified this correctly using Cross Validation

**Cross Validation: Why CV matters**

Tests model on multiple unseen splits

Detects if model just memorized data